elementary binary operations that can be handled directly within the CPU. While this provides the same result, the execution time is much slower, say, a factor of 10 to 20.

Most personal computer software can be used with or without a math coprocessor. This is accomplished by having the compiler generate machine code to handle both cases, all stored in the final executable program. If a math coprocessor is present on the particular computer being used, one section of the code will be run. If a math coprocessor is not present, the other section of the code will be used. The compiler can also be directed to generate code for only one of these situations. For example, you will occasionally find a program that requires that a math coprocessor be present, and will crash if run on a computer that does not have one. Applications such as word processing usually do not benefit from a math coprocessor. This is because they involve moving data around in memory, not the calculation of mathematical expressions. Likewise, calculations involving fixed point variables (integers) are unaffected by the presence of a math coprocessor, since they are handled within the CPU. On the other hand, the execution speed of DSP and other computational programs using floating point calculations can be an order of magnitude different with and without a math coprocessor.

The CPU and main memory are contained in separate chips in most computer systems. For obvious reasons, you would like the main memory to be very large and very fast. Unfortunately, this makes the memory very expensive. The transfer of data between the main memory and the CPU is a very common bottleneck for speed. The CPU *asks* the main memory for the binary information at a particular memory address, and then must *wait* to receive the information. A common technique to get around this problem is to use a **memory cache**. This is a small amount of very fast memory used as a buffer between the CPU and the main memory. A few hundred kilobytes is typical. When the CPU requests the main memory to provide the binary data at a particular address, high speed digital electronics copies a *section* of the main memory around this address into the memory cache. The next time that the CPU requests memory information, it is very likely that it will already be contained in the memory cache, making the retrieval very rapid. This is based on the fact that programs tend to access memory locations that are nearby neighbors of previously accessed data. In typical personnel computer applications, the addition of a memory cache can improve the overall speed by several times. The memory cache may be in the same chip as the CPU, or it may be an external electronic device.

The rate at which data can be transferred between subsystems depends on the number of parallel data lines provided, and the maximum rate that digital signals that can be passed along each line. Digital data can generally be transferred at a much higher rate within a single chip as compared to transferring data between chips. Likewise, data paths that must pass through electrical connectors to other printed circuit boards (i.e., a bus structure) will be slower still. This is a strong motivation for stuffing as much electronics as possible inside the CPU.